

# Sequential File Organization

## Definition

### Key Idea

Sequential file organization is a method where records are stored in **sorted order** based on a **search key**. This method is particularly suitable for applications that require sequential processing of an entire file or range queries.

## Characteristics

- Records are stored in **order of a search-key attribute**.
- Sequential processing is highly efficient (e.g., payroll generation, report creation).
- Random access is possible, but less efficient than indexed files.

## Record Operations

### Insertion

- Locate the position where the new record should be inserted.
- If there is free space in the block, insert the record there.
- If the block is full, place the record in an **overflow block**.
- Update pointer chains to maintain logical order.

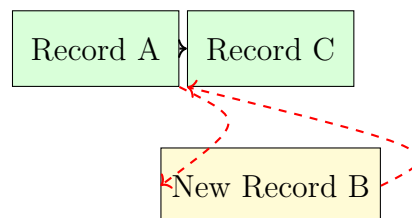


Figure 1: Insertion using Overflow Block (New Record B)

## Deletion

- Records are typically removed using a **pointer chain** to mark deleted entries.
- Actual space reclamation may require file reorganization.

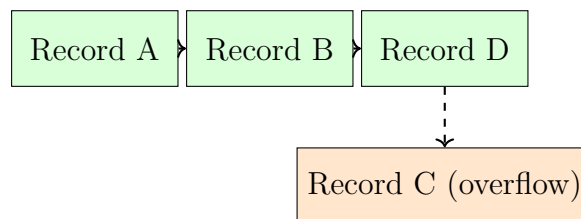


Figure 2: Deletion using Pointer Chain (Record B removed)

## Reorganization

- From time to time, the file must be reorganized to:
  - Restore sequential ordering.
  - Merge overflow records back into main blocks.
  - Reduce fragmentation.

Before Reorganization



After Reorganization

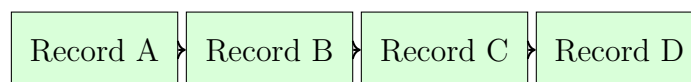


Figure 3: Reorganization merges overflow records back into main blocks

## Advantages

- Efficient for sequential processing of the entire file.
- Ideal for range queries.
- Good for applications like payroll systems, transaction logs, and report generation.

## Disadvantages

- Insertions and deletions are costly due to reorganization and overflow handling.
- Requires periodic reorganization to maintain efficiency.
- Not suitable for applications with frequent random updates.

## Use Cases

- Banking systems (monthly statement generation).
- Payroll systems (sequential salary processing).
- Large reporting systems where data is processed in sorted order.

## Summary

### Summary

- Records are stored in order based on a search key.
- Efficient for sequential access and range queries.
- Insertions and deletions are complex due to overflow blocks and reorganization needs.
- Best suited for applications where sequential file scans dominate.